

SQL Queries

Final queries for E3, M4 and H4

Round 1, Phase 2: Analytical Core

Competition: Data Vortex | AARUUSH'26
Theme: Rebuilding the Social Engine
Team: Entropy
Members: Saanvi Grover & Aditya Sharma

Questions: E3 · M4 · H4
Database: SQLite (Entropy_social_engine.db)

Database Schema

All three queries run against a two-table SQLite schema built from the Phase 1 cleaned CSVs. Constraints live in the DDL, so the engine enforces them rather than relying on the cleaning code.

```
1 PRAGMA foreign_keys = ON;
2
3 CREATE TABLE users (
4     user_id        TEXT PRIMARY KEY,
5     location       TEXT NOT NULL,
6     language       TEXT NOT NULL,
7     account_created DATE NOT NULL,
8     follower_count INTEGER NOT NULL CHECK (follower_count >= 0)
9 );
10
11 CREATE TABLE posts (
12     post_id        TEXT PRIMARY KEY,
13     user_id        TEXT NOT NULL REFERENCES users(user_id),
14     platform       TEXT, -- NULL where corrupted (1,784 posts)
15     text_content   TEXT, -- NULL where corrupted
16     timestamp      DATETIME NOT NULL,
17     likes          INTEGER CHECK (likes >= 0), -- NULL where corrupted (1,814 posts)
18     shares         INTEGER NOT NULL CHECK (shares >= 0),
19     comments       INTEGER NOT NULL CHECK (comments >= 0)
20 );
21
22 CREATE INDEX idx_posts_user_id ON posts(user_id);
23 CREATE INDEX idx_posts_platform ON posts(platform);
24 CREATE INDEX idx_posts_timestamp ON posts(timestamp);
25 CREATE INDEX idx_users_location ON users(location);
26 CREATE INDEX idx_users_language ON users(language);
27
28 CREATE VIEW user_posts AS
29 SELECT p.*, u.location, u.language, u.account_created, u.follower_count,
30        p.likes + p.shares + p.comments AS total_engagement
31 FROM posts p JOIN users u ON u.user_id = p.user_id;
```

Row counts: users = 1,500; posts = 12,000.

Constraints are enforced by the engine

Each statement below was executed against the database and rejected by SQLite:

Attempted insert	SQLite response
Post for a user_id that does not exist	FOREIGN KEY constraint failed
Post with likes = -5	CHECK constraint failed: likes >= 0
Post with shares = NULL	NOT NULL constraint failed: posts.shares

E3 – Average Engagement by Platform (Easy)

Challenge. Calculate the average likes, shares, and comments for each platform. Which platform generates the highest average total engagement?

```
1 -- E3: Average Engagement by Platform
2 -- Calculate the average likes, shares and comments for each platform.
3 -- Which platform generates the highest average total engagement?
4 --
5 -- Only posts with a missing platform are excluded, as the question implies.
6 -- AVG() skips NULLs, so avg_likes uses posts that have a like count while
7 -- avg_shares / avg_comments use every post on the platform. For total
```

```

8  -- engagement, likes + shares + comments is NULL when likes is missing, so
9  -- AVG() drops those posts: exactly the "ignore posts where likes are missing"
10 -- rule the brief uses to define total engagement (E2).
11
12 WITH platform_stats AS (
13     SELECT
14         platform,
15         COUNT(*)                AS posts,
16         COUNT(likes)            AS posts_with_likes,
17         ROUND(AVG(likes), 1)    AS avg_likes,
18         ROUND(AVG(shares), 1)   AS avg_shares,
19         ROUND(AVG(comments), 1) AS avg_comments,
20         ROUND(AVG(likes + shares + comments), 1) AS avg_total_engagement
21     FROM posts
22     WHERE platform IS NOT NULL
23     GROUP BY platform
24 ),
25 overall AS (
26     SELECT AVG(likes + shares + comments) AS grand_avg
27     FROM posts
28     WHERE platform IS NOT NULL
29 )
30 SELECT
31     RANK() OVER (ORDER BY ps.avg_total_engagement DESC) AS engagement_rank,
32     ps.platform,
33     ps.posts,
34     ps.posts_with_likes,
35     ps.avg_likes,
36     ps.avg_shares,
37     ps.avg_comments,
38     ps.avg_total_engagement,
39     ROUND(100.0 * (ps.avg_total_engagement - o.grand_avg) / o.grand_avg, 2)
40     AS pct_vs_overall
41 FROM platform_stats ps
42 CROSS JOIN overall o
43 ORDER BY engagement_rank;

```

Result (5 rows):

Rank	Platform	Posts	With likes	Avg likes	Avg shares	Avg comments	Avg total	vs overall
1	Instagram	1,989	1,693	2,500.9	1,040.8	499.8	4,040.0	+0.78%
2	YouTube	2,073	1,747	2,517.8	1,011.8	504.4	4,031.8	+0.58%
3	Facebook	2,074	1,755	2,528.9	984.2	506.9	4,016.0	+0.18%
4	Reddit	2,031	1,742	2,488.1	1,002.2	511.2	4,003.9	-0.12%
5	Twitter	2,049	1,725	2,437.7	1,005.4	506.1	3,951.9	-1.42%

Answer: Instagram generates the highest average total engagement (4,040.0 per post).

Query plan (index use): SEARCH posts USING INDEX idx_posts_platform.

M4 – Platform Behaviour by High-Follower Users (Medium)

Challenge. Among users with at least 30,000 followers, determine which platform gives them the highest average engagement per post.

```

1  -- M4: Platform Behaviour by High-Follower Users
2  -- Among users with at least 30,000 followers, which platform gives them the
3  -- highest average engagement per post?
4  --
5  -- The high-follower cohort is compared with users BELOW 30,000 followers

```

```

6  -- (not with all users, which would include the cohort itself and shrink any
7  -- gap). Conditional aggregation computes both groups in a single scan, and
8  -- ranking each group separately shows whether the platform order holds.
9  -- Engagement follows the brief's definition: posts with missing likes are
10 -- ignored.
11
12 WITH post_engagement AS (
13     SELECT
14         p.platform,
15         p.user_id,
16         u.follower_count >= 30000          AS is_high_follower,
17         p.likes + p.shares + p.comments    AS engagement
18     FROM posts p
19     JOIN users u ON u.user_id = p.user_id
20     WHERE p.platform IS NOT NULL
21           AND p.likes IS NOT NULL
22 ),
23 by_platform AS (
24     SELECT
25         platform,
26         COUNT(DISTINCT CASE WHEN is_high_follower THEN user_id END) AS high_follower_users,
27         SUM(is_high_follower)                                     AS high_follower_posts
28     ,
29         AVG(CASE WHEN is_high_follower THEN engagement END)      AS avg_high,
30         AVG(CASE WHEN NOT is_high_follower THEN engagement END)  AS avg_other
31     FROM post_engagement
32     GROUP BY platform
33 )
34 SELECT
35     RANK() OVER (ORDER BY avg_high DESC) AS rank_high_followers,
36     RANK() OVER (ORDER BY avg_other DESC) AS rank_other_users,
37     platform,
38     high_follower_users,
39     high_follower_posts,
40     ROUND(avg_high, 1) AS avg_eng_high_followers,
41     ROUND(avg_other, 1) AS avg_eng_other_users,
42     ROUND(100.0 * (avg_high - avg_other) / avg_other, 2) AS lift_pct
43 FROM by_platform
44 ORDER BY rank_high_followers;

```

Result (5 rows):

Rank 30k+	Rank <30k	Platform	30k+ users	30k+ posts	Avg 30k+	Avg <30k	Lift
1	4	Instagram	404	678	4,145.2	3,969.7	+4.42%
2	2	Facebook	404	674	4,000.9	4,025.4	-0.61%
3	3	Reddit	408	677	3,981.4	4,018.3	-0.92%
4	5	Twitter	394	664	3,978.5	3,935.2	+1.10%
5	1	YouTube	419	715	3,947.4	4,090.4	-3.50%

Answer: for the 594 users with at least 30,000 followers, Instagram gives the highest average engagement per post (4,145.2).

Query plan (index use): SEARCH p USING INDEX idx_posts_platform; SEARCH u USING INDEX sqlite_autoindex_users_1 (primary key).

H4 – Follower-to-Engagement Anomaly (Hard)

Challenge. Find users who have fewer than 5,000 followers but whose total post engagement places them among the top 10% of all users.

```

1  -- H4: Follower-to-Engagement Anomaly
2  -- Users with fewer than 5,000 followers whose total post engagement places
3  -- them in the top 10% of all users.
4  --
5  -- Level 1 impute: a missing like count is replaced by that user's own
6  --                  average likes (global average if they have none), so a
7  --                  corrupted value neither erases a post nor penalises
8  --                  whichever users happened to be corrupted more often.
9  -- Level 2 aggregate: roll posts up to users (total and per-post engagement).
10 -- Level 3 rank: decile every user twice, by total and by per-post
11 --                engagement; user_id breaks ties so results are
12 --                deterministic.
13 -- Level 4 filter: top total decile AND fewer than 5,000 followers, then
14 --                  classify each survivor by whether their per-post
15 --                  engagement is also top-decile.
16
17 WITH user_like_avg AS (
18     SELECT user_id, AVG(likes) AS avg_likes
19     FROM posts
20     GROUP BY user_id
21 ),
22 global_like_avg AS (
23     SELECT AVG(likes) AS avg_likes FROM posts
24 ),
25 post_engagement AS (
26     SELECT
27         p.user_id,
28         p.likes IS NULL AS likes_imputed,
29         COALESCE(p.likes, ula.avg_likes, g.avg_likes) + p.shares + p.comments AS engagement
30     FROM posts p
31     JOIN user_like_avg ula ON ula.user_id = p.user_id
32     CROSS JOIN global_like_avg g
33 ),
34 user_engagement AS (
35     SELECT
36         u.user_id,
37         u.location,
38         u.follower_count,
39         COUNT(*) AS post_count,
40         SUM(pe.likes_imputed) AS imputed_posts,
41         SUM(pe.engagement) AS total_engagement,
42         AVG(pe.engagement) AS avg_engagement_per_post
43     FROM users u
44     JOIN post_engagement pe ON pe.user_id = u.user_id
45     GROUP BY u.user_id, u.location, u.follower_count
46 ),
47 ranked_users AS (
48     SELECT
49         *,
50         NTILE(10) OVER (ORDER BY total_engagement DESC, user_id) AS total_decile,
51         NTILE(10) OVER (ORDER BY avg_engagement_per_post DESC, user_id) AS per_post_decile
52     ,
53     PERCENT_RANK() OVER (ORDER BY total_engagement DESC) AS pct_rank
54     FROM user_engagement
55 )
56 SELECT
57     RANK() OVER (ORDER BY total_engagement DESC) AS anomaly_rank,
58     user_id,
59     location,
60     follower_count,
61     post_count,
62     imputed_posts,
63     ROUND(total_engagement, 0) AS total_engagement,
64     ROUND(avg_engagement_per_post, 1) AS avg_eng_per_post,

```

```

64     ROUND(100.0 * pct_rank, 2)                AS top_pct_of_users,
65     per_post_decile,
66     CASE WHEN per_post_decile = 1
67         THEN 'Exceptional per post'
68         ELSE 'Volume-driven' END              AS classification
69 FROM ranked_users
70 WHERE total_decile = 1      -- top 10% of all users by total engagement
71     AND follower_count < 5000 -- ...despite a small audience
72 ORDER BY total_engagement DESC;

```

Result (16 rows; top-10% threshold = 48,159 total engagement):

#	User	Location	Followers	Posts	Imputed	Total	Per post	Top %	Per-post decile	Classification
1	user_uerv85na	Rome, Italy	1,824	16	3	73,544	4,596.5	0.20	2	Volume-driven
2	user_hdas0iau	Rio de Janeiro, Brazil	1,620	16	2	63,346	3,959.1	1.07	6	Volume-driven
3	user_fgjkkrie	Lyon, France	2,211	14	0	62,690	4,477.9	1.13	3	Volume-driven
4	user_n0ok02rt	Dubai, UAE	2,531	18	4	60,876	3,382.0	1.53	9	Volume-driven
5	user_6wra58f7	Johannesburg, South Africa	4,953	14	1	57,314	4,093.8	3.54	5	Volume-driven
6	user_5oe5t3js	London, UK	3,151	13	2	56,063	4,312.5	3.94	4	Volume-driven
7	user_ogtvuuki	Cairo, Egypt	4,459	11	3	54,865	4,987.7	4.60	1	Exceptional per post
8	user_r7eglrac	Houston, USA	2,052	12	0	54,552	4,546.0	4.74	2	Volume-driven
9	user_rrluzkql	Milan, Italy	1,069	14	2	54,495	3,892.5	4.80	6	Volume-driven
10	user_67hyf45u	Vancouver, Canada	898	11	1	52,673	4,788.4	6.14	2	Volume-driven
11	user_cqaitbuv	Johannesburg, South Africa	3,382	11	1	52,237	4,748.8	6.60	2	Volume-driven
12	user_knjtsbf3	Toronto, Canada	1,298	11	2	51,005	4,636.8	7.74	2	Volume-driven
13	user_zozhwbis	Paris, France	4,580	11	2	50,747	4,613.3	8.21	2	Volume-driven
14	user_zwtxkrs	Houston, USA	1,009	13	4	48,785	3,752.7	9.21	7	Volume-driven
15	user_0aacyvtz	Rome, Italy	1,452	11	1	48,499	4,409.0	9.61	3	Volume-driven
16	user_kbdvf8d6	Tokyo, Japan	4,645	10	0	48,490	4,849.0	9.67	1	Exceptional per post

Answer: 16 users with fewer than 5,000 followers are in the top 10% by total engagement. Two of them (user_ogtvuuki, user_kbdvf8d6) are also in the top 10% per post.

Query plan (index use): SCAN posts USING INDEX idx_posts_user_id (per-user like averages); SEARCH p USING INDEX idx_posts_user_id; SEARCH u USING INDEX sqlite_autoindex_users_1.

Reproducibility

- Engine: SQLite 3 (window functions require version 3.25 or later), run through Python's `sqlite3` and `pandas` in `Entropy_01_SQL_Analysis.ipynb`, which also rebuilds the database and runs `EXPLAIN QUERY PLAN` for each query.
- Each query is stored verbatim in `queries/`. The three output screenshots in `images/` are captures of Jupyter executing these exact files.
- Every query is a single read-only `SELECT`; rankings use `user_id` as a tie-break, so results are deterministic.